

STA521

Predictive Modelling and Statistical Learning

Lecture 21: Decision Trees + Bagging

Yuansi Chen

Fall 2022

Duke University

<https://www2.stat.duke.edu/courses/Fall22/sta521.001/>



1. Decision tree basics
 - CART
2. Bagging

Decision trees: basic ideas

Given $\mathbf{X} \in \mathbb{R}^{n \times p}$, $\mathbf{y}$

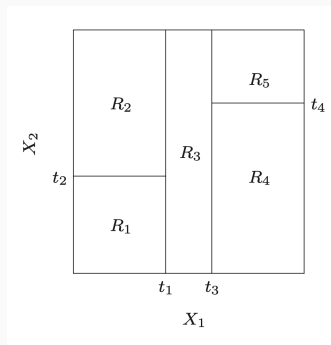
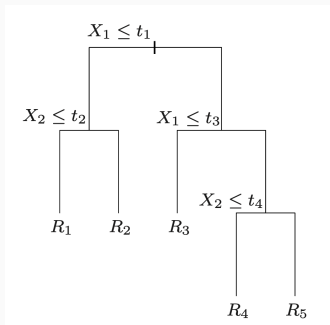
- A decision tree divides up the covariate space into rectangles and then fit a very simple model in each rectangle
- It works both when y is discrete and continuous, i.e., both for **classification** and **regression**
- Rectangles can be achieved by making successive binary splits on the predictors $\mathbf{X}_1, \dots, \mathbf{X}_p$ via the divides like

$$X_j \leq c \text{ or } X_j > c$$

Recall: classification trees

Classification trees divide the feature space $\mathbb{R}^p$ up into several rectangles, and then assign to each rectangle R_j a particular predicted class c_j :

$$\hat{f}^{\text{tree}}(x) = \sum_{j=1}^m c_j \cdot \mathbf{1}_{\{x \in R_j\}}$$



Three major issues

- Which predictor to split?
- How to choose the splits?
- How big to grow the tree?

The CART algorithm provides a set of rules to build trees (Breiman et al. (1984))

- **Top-down split order:** root split first, then second level, etc.
- **Greedy split at each node:** choose the split to achieve the biggest drop in misclassification error (or other impurity measures)
- **Depth strategy:** grow a large tree and then **prune** at the end (instead of early stopping)

At the root level, CART split on variable j with split threshold s , defining two regions

$$R_1 = \{x \in \mathbb{R}^p : x_j \leq s\}, R_2 = \{x \in \mathbb{R}^p : x_j > s\}$$

CART more details

At the root level, CART split on variable j with split threshold s , defining two regions

$$R_1 = \{x \in \mathbb{R}^p : x_j \leq s\}, R_2 = \{x \in \mathbb{R}^p : x_j > s\}$$

It greedily chooses j, s by minimizing the misclassification error

$$\arg \min_{j,s} ([1 - \hat{p}_{c_1}(R_1)] + [1 - \hat{p}_{c_2}(R_2)])$$

where c_1 is the most common class in R_1 and c_2 is the most common class in R_2

Then repeat at each node.

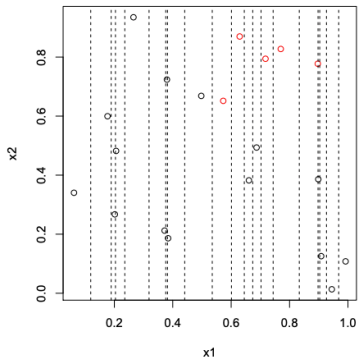
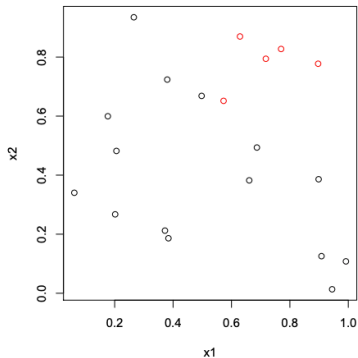
How do we find the best split threshold s ?

It seems that s can be any real number between $(-\infty, \infty)$

How do we find the best split threshold s ?

It seems that s can be any real number between $(-\infty, \infty)$

But in practice, once variable j is decided, we only need consider $n_m - 1$ splits (where n_m is the number of data points in the region to be split)



How do we prune a large tree?

Prunning

Starting from a tree, we collapse some of its leaves into the parent nodes

How do we prune a large tree?

Prunning

Starting from a tree, we collapse some of its leaves into the parent nodes

For any tree T , denote $|T|$ its number of leaves. Define

$$C_{\alpha}(T) = \sum_{j=1}^{|T|} [1 - \hat{p}_{G_j}(R_j)] + \alpha |T|$$

We seek the tree T a sub-tree of the original T_0 that minimizes $C_{\alpha}(T)$ is minimized

- Can be done by pruning one leaf at a time
- α chosen via cross-validation

rpart or **tree** in R. (**rpart** does cross-validation automatically)

- Classification or regression? Both are fine
- Continuous or discrete X ? Both are fine
- Center and Scale? Not really needed

Decision trees on spam dataset

spam dataset: $n = 4601$ emails, of which 1813 are considered spam. For each email we have $p = 58$ attributes. The first 54 measure the frequencies of 54 key words or characters (e.g., “free”, “need”, “edu”). The last 3 measure

- the average length of uninterrupted sequences of capitals
- the length of the longest uninterrupted sequence of capitals
- the sum of lengths of uninterrupted sequences of capitals

(from ESL 9.2.5)

Cross-validation error curve for the spam data

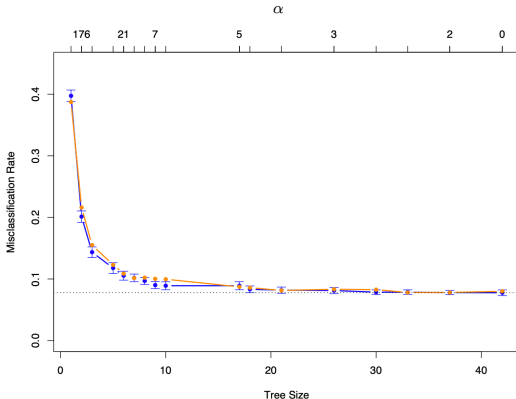
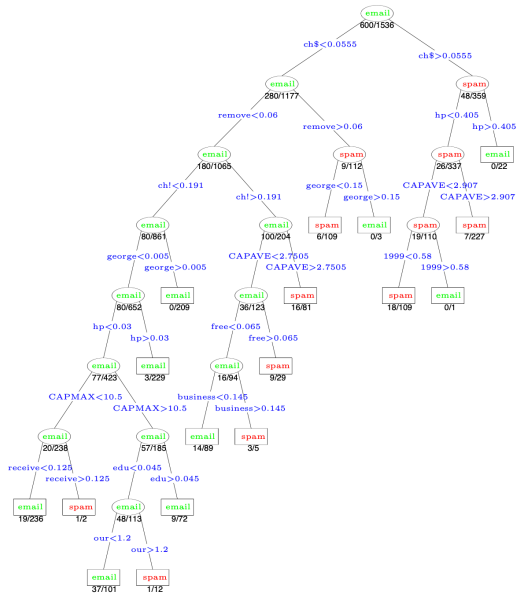


FIGURE 9.4. Results for `spam` example. The blue curve is the 10-fold cross-validation estimate of misclassification rate as a function of tree size, with standard error bars. The minimum occurs at a tree size with about 17 terminal nodes (using the “one-standard-error” rule). The orange curve is the test error, which tracks the CV error quite closely. The cross-validation is indexed by values of α , shown above. The tree sizes shown below refer to $|T_\alpha|$, the size of the original tree indexed by α .

Tree depth = 17, chose by CV



Other impurity measures

- **misclassification error:** for region R_j

$$1 - \hat{p}_{c_j}(R_j)$$

- **Gini index:**

$$\sum_{k=1}^K \hat{p}_k(R_j)[1 - \hat{p}_k(R_j)]$$

- **Cross-entropy or deviance:**

$$-\sum_{k=1}^K \hat{p}_k(R_j) \log(\hat{p}_k(R_j))$$

Comparison of impurity measures

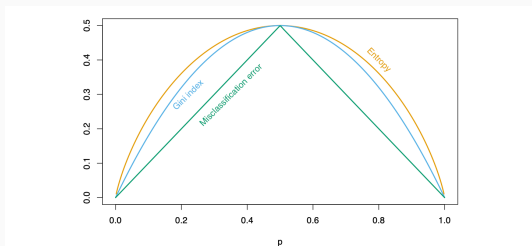


FIGURE 9.3. Node impurity measures for two-class classification, as a function of the proportion p in class 2. Cross-entropy has been scaled to pass through $(0.5, 0.5)$.

- Misclassification error is what we try to optimize in the end, so misclassification error is usually preferable
- Viewing other measures as smoothed version of misclassification error could make the optimization of the splits easier if more sophisticated way of splitting is used.
- They are similar in general

How about regression trees?

Two modifications needed

- Prediction at the leaf
- Split rule: what replaces misclassification error?

Prediction at the leaf

- Instead of

$$\hat{f}^{\text{tree}}(x) = \sum_{j=1}^m c_j \mathbf{1}_{x \in R_j}$$

- Use the average response of all points in the region

$$c_j = \frac{1}{n_j} \sum_{x_i \in R_j} y_i$$

Can be extended to running linear regression at leaf

Split rule

- use squared error loss instead of misclassification error

How good are trees empirically?

Not great!

Bias variance trade-off in general

- Shallow trees have large bias
- Deep trees have large variance

Trees optimized greedily generally suffer from high variance

because they are quite instable:

- a smaller change in the observed data can lead to a dramatically different sequence of splits

This instability comes from their hierarchical nature: once a split is made, it is permanent and can never be “unmade” further down in the tree

Recall: pros and cons of CART

Pros

- “model-free”
- interpretable decision rules
- handle qualitative predictors
- fast compared to k-NN

Cons

- If training error is optimized, then trees usually have high variance
- Non-robust to perturbations. The separations made by splits are enforced at all lower levels of the tree, which means that if the data is perturbed slightly, the new tree can have a considerably different sequence of splits, leading to a different classification rule

Tree vs. linear models

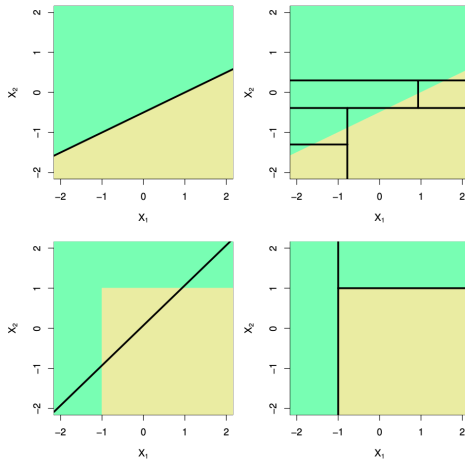


FIGURE 8.7. Top Row: A two-dimensional classification example in which the true decision boundary is linear, and is indicated by the shaded regions. A classical approach that assumes a linear boundary (left) will outperform a decision tree that performs splits parallel to the axes (right). Bottom Row: Here the true decision boundary is non-linear. Here a linear model is unable to capture the true decision boundary (left), whereas a decision tree is successful (right).

Are there ways to control the variance or stabilize the tree predictions?

Bagging

The bootstrap (Efron 1979) is a fundamental resampling tool in statistics.

- **Idea of bootstrap:** we can estimate the properties of the true distribution $\mathcal{F}$ by resampling of the empirical distribution $\hat{\mathcal{F}}$.

Empirical distribution:

Given training data $(x_i, y_i), i = 1, \dots, n$, the empirical distribution $\hat{\mathcal{F}}$ is simply a discrete distribution putting equal weight $\frac{1}{n}$ on each training point

A bootstrap sample

A bootstrap sample of size m from the training data is

$$(x_i^*, y_i^*), i = 1, \dots, m$$

where each (x_i^*, y_i^*) are drawn uniformly at random from $(x_1, y_1), \dots, (x_n, y_n)$ **with replacement**

- Idea: what a bootstrap sample is to the empirical sample approximately “mimics” what the empirical sample is to the true distribution

Not all points are represented in a bootstrap sample

When $m = n$, about 36% of points are left out, for large n

For each bootstrap sample of size n , we can compute a classification tree classifier. **Bagging**

- averages the predictions from the classification trees over a collection of B bootstrap samples
- to classify a new point $x \in \mathbb{R}^p$, we take the most common predicted class

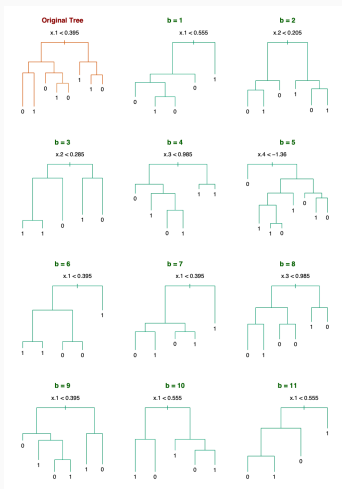
$$\hat{f}^{\text{bag}}(x) = \arg \max_{k=1,\dots,K} \sum_{b=1}^B \mathbf{1}_{\{\hat{f}^{\text{tree},b}(x)=k\}}$$

also called the majority vote or the consensus strategy

Bagging on simulated data

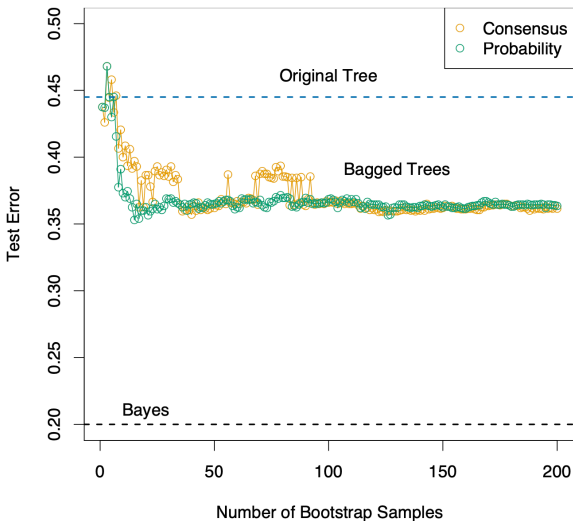
Example from ESL 8.7.1: $n = 30$ training data points, $p = 5$ features, and $K = 2$ classes. The response Y was generated according to

$$\mathbb{P}(Y = 1 | x_1 \leq 0.5) = 0.2, \mathbb{P}(Y = 1 | x_1 > 0.5) = 0.8$$



Bagging helps on simulated data

Bagging helps decrease the misclassification rate



An alternative way to aggregate: via probability

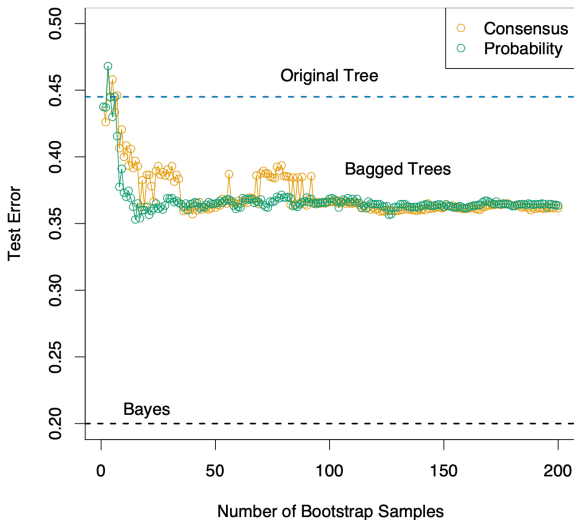
Bagging the class probability: for each tree, we look at its predicted class probability $\hat{p}_k^{\text{tree},b}$, $k = 1, \dots, K$. Then we define the bagging estimate of class probabilities

$$\hat{p}_k^{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{p}_k^{\text{tree},b}(x)$$

The final bagging classifier chooses the class with the highest probability

$$\hat{f}^{\text{bag, alt}}(x) = \arg \max_{k=1, \dots, K} \hat{p}_k^{\text{bag}}(x)$$

Bagging via probability sometimes performs better



Why is bagging working?

How much does the cow weight?

- A 1906 county fair in Plymouth, England had a contest to guess the weight of an ox. Francis Galton calculated the median of everyone's guesses. The median guess was 1,207 pounds, and the true weight was 1,198 pounds, so the error was less than 1%

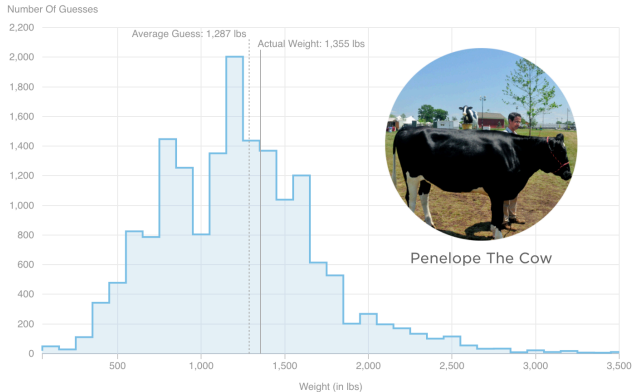
Intuition 1: the wisdom of the crowds

How much does the cow weight?

- A 1906 county fair in Plymouth, England had a contest to guess the weight of an ox. Francis Galton calculated the median of everyone's guesses. The median guess was 1,207 pounds, and the true weight was 1,198 pounds, so the error was less than 1%
- NPR repeated the experiment in 2015 with a cow named Penelope whose photo they published online. They got 17,000 guesses, and the average guess was 1,287 pounds. Penelope's actual weight was 1,355 pounds

Sometimes the average opinion of non-experts can be very good

How much does the cow weight?



Source: The Internet.

Credit: Quoctrung Bui/NPR

Intuition 2: bagging independent classifiers

Suppose of we have B independent classifiers, each with misclassification rate 0.4. Assume the true label at x is 1, then

$$\mathbb{P}(\hat{f}^b(x) = 0) = 0.4$$

The bagged classifier is

$$\hat{f}^{\text{bag}}(x) = \arg \max_{k=0,1} \sum_{b=1}^B \mathbf{1}_{\{\hat{f}^b(x)=k\}}$$

Let $V_0 = \sum_{b=1}^B \mathbf{1}_{\{\hat{f}^b(x)=0\}}$ be the number of votes for class 0, it is distributed

$$V_0 \sim \text{Binom}(B, 0.4)$$

Thus the misclassification rate of the bagged classifier is

$$\mathbb{P}(\hat{f}^{\text{bag}}(x) = 0) = \mathbb{P}(V_0 \geq B/2)$$

which tends to 0 when B is large.

Let $V_0 = \sum_{b=1}^B \mathbf{1}_{\{\hat{f}^b(x)=0\}}$ be the number of votes for class 0, it is distributed

$$V_0 \sim \text{Binom}(B, 0.4)$$

Thus the misclassification rate of the bagged classifier is

$$\mathbb{P}(\hat{f}^{\text{bag}}(x) = 0) = \mathbb{P}(V_0 \geq B/2)$$

which tends to 0 when B is large.

Do we have independent classifiers when we do bagging?

Do we have independent classifiers when we do bagging?

No.

- $\hat{f}^{\text{tree},b}$ are clearly not independent, because they are fit on very similar data
- They are likely to be positively correlated
- Since about 36% data points are not present each bootstrap (see HW), they are not perfectly correlated.

Does bagging always improve?

No. In the simplified example of bagging independent classifiers, if we assume each classifier has a misclassification rate 0.6 instead. Then by the same arguments, the bagged classifier has a misclassification rate of

$$\mathbb{P}(\hat{f}^{\text{bag}}(x) = 0) = \mathbb{P}(V_0 \geq B/2)$$

As $V_0 \sim \text{Binom}(B, 0.6)$, tends to 1 when B is large.

bagging bad classifiers can also degrade accuracy!

Summary of why bagging works

- Bootstrap provides resamples of the dataset which approximately “mimics” new datasets
- The bootstrap samples are not identical but not completely independent either, so averaged classifier reduces some variance.
- Each classifier is not too bad, so overall the reduced variance help improve accuracy

Bagging: pros and cons

Pros:

- better prediction/classification performance
- more stable

Cons:

- Loss of straightforward interpretability
- computational complexity grows with B

The more independent each bootstrap classifier is, the better the variance reduction works

Can we do better in creating more independent classifiers to average over?

Coming next: random forests

- Bagging
 - draws many bootstrap-sampled data sets from the original training data
 - train on each sampled data set individually
 - then aggregate predictions at the end
- Two ways to aggregate: the consensus strategy and the probability strategy

Thank you

